

```
In [1]: %matplotlib inline
import jax.numpy as jnp
from jax import random
import matplotlib.pyplot as plt
import seaborn as snb

from scipy.stats import poisson
from scipy.stats import gamma

from exercise9 import compute_effective_sample_size
from exercise9 import compute_Rhat
from exercise9 import plot_summary
from exercise9 import metropolis

# we want to use 64 bit floating precision
import jax
jax.config.update("jax_enable_x64", True)

# style stuff
snb.set_style('darkgrid')
snb.set_theme(font_scale=1.1)
```

02477 Bayesian Machine Learning - Exercise 9

Overview

Markov Chain Monte Carlo (MCMC) techniques like Metropolis-Hastings and Gibbs sampling are very powerful methods in machine learning and statistics as they provide the computational tools for generating samples from complex distributions, e.g. posterior distributions of complex non-conjugate Bayesian models, but they are by no means limiting the Bayesian models.

The purpose of this exercise is to become familiar with Gibbs sampling as well as the Markov Chain Monte Carlo convergence diagnostics. First, we will study how to detect convergence in practice and how to compute the effective number of samples by taking the autocorrelation of the MCMC into account. Then, we will study Gibbs sampling using a simple Bayesian model for change point detection.

Content

- Part 1: The $\hat{R}$ -statistic and the effective sample size
- Part 2: A simple model for change point detection in time series using Gibbs sampling
- Part 3: Implementing the Gibbs sampler
- Part 4: Visualizing the posterior distributions for the change point detection model

Part 1: The $\hat{R}$ -statistic and the effective sample size

The purpose of the first part is to become familiar with the MCMC convergence diagnostics denoted $\hat{R}$ as well as the effective sample size S_{eff} . MCMC theory states that a well-designed MCMC algorithm converges asymptotically to the true target distribution, meaning that the Markov chain eventually will converge to the target distribution. The downside is that there is no strict theory for predicting when a chain will reach its stationary distribution, but there do exist heuristics that work very well in practice.

Assessing convergence

Intuitively, we can test convergence by running **multiple chains** in parallel starting from **different initial conditions**. After a given number of iterations, we can compare the chains. If the chains have reached stationarity, then they should all have the same stationary distribution independent of the initial condition. Therefore, if our chains do not have the same distributions, we can conclude the chains have not reached stationarity yet. In this case, we say that the chains have not **mixed**.

In practice, the so-called $\hat{R}$ -statistic, also called the **potential scale reduction factor**, works well for assessing convergence:

$$\hat{R}^2 = \frac{S-1}{S} + \frac{1}{S} \frac{B}{W},$$

where S is the number of samples/iterations, B is the **between-chain variance** and W is the **within-chain variance**.

If the between-chain variance is equal to the within-chain variance ($B \approx W$) the chains look alike and then then $\hat{R} = 1$, otherwise $\hat{R} > 1$. For most applications, the threshold $\hat{R} < 1.1$ works well for assessing convergence, but for more critical applications the threshold $\hat{R} < 1.01$ is recommended.

For more details, see Section 12.6.2.2. in Murphy2 or see p. 284 in Bayesian Data Analysis (<http://www.stat.columbia.edu/~gelman/book/BDA3.pdf>).

Effective sample size

MCMC draws are autocorrelated, so $S_{\text{tot}} = MS$ correlated draws contain less information than S_{tot} i.i.d. draws. The effective sample size (ESS) adjusts for this:

$$S_{\text{eff}} = \frac{S}{\sum_{t=-\infty}^{\infty} \rho_t} = \frac{S}{1 + 2 \sum_{t=1}^{\infty} \rho_t},$$

where ρ_t is the autocorrelation of the chains at time lag t . Informally, a set of S MCMC samples contains less information than S independent samples from the same distribution due to the correlation between MCMC samples. The effective sample size helps us correct for this when computing the MCSE.

ESS directly control Monte Carlo error for the mean

$$\text{MCSE}(\hat{\mu}) = \sqrt{\frac{\overline{V(\hat{\theta})}}{S_{\text{eff}}}}$$

The provided functions `compute_Rhat` and `compute_effective_sample_size` compute $\hat{R}$ and S_{eff} respectively.

These diagnostics are evaluated after removing the **warmup** samples. And after we computed the diagnostics, we **merge** the samples from the individual chains into one long chain before using the samples for inference purposes.

WORKFLOW NOTE: Compute diagnosis **after removing warmup samples**. Use $\hat{R}$ and S_{eff} for assessing convergence/mixing. Only then **merge** chains into one long chain for posterior summaries.

A bi-modal toy distribution

To build intuition, we will first investigate these two statistics for a simple toy distribution, where we can easily compute the mean and variance reference:

$$p(x) = \frac{1}{2}N(x \mid -3, 4) + \frac{1}{2}N(x \mid 1, 2)$$

Computing the first two moments yields (if you don't know how to do this, ask the teachers):

$$E[x] = \frac{1}{2} \cdot (-3) + \frac{1}{2} \cdot 1 = -1$$

$$E\left[x^2\right] = \frac{1}{2} \cdot ((-3)^2 + 4) + \frac{1}{2} \cdot (1^2 + 2) = 8,$$

and hence, the variance of $p(x)$ is

$$V[x] = E\left[x^2\right] - E[x]^2 = 8 - (-1)^2 = 7.$$

These references let you **sanity-check MCMC output**:

- With **good mixing across both modes**, you should recover:
 - Mean ≈ -1
 - Variance ≈ 7
 - $\hat{R} \approx 1$
 - Large Seff (effective sample size)

- If different chains **stick in different modes**:

- Between-chain variance $B \gg W$
- $\hat{R} > 1$

- If all chains **get trapped in the same mode**:

- $\hat{R}$ may **look fine**
- But Seff will remain **small**

☞ Hence, it's important to inspect **both diagnostics**:

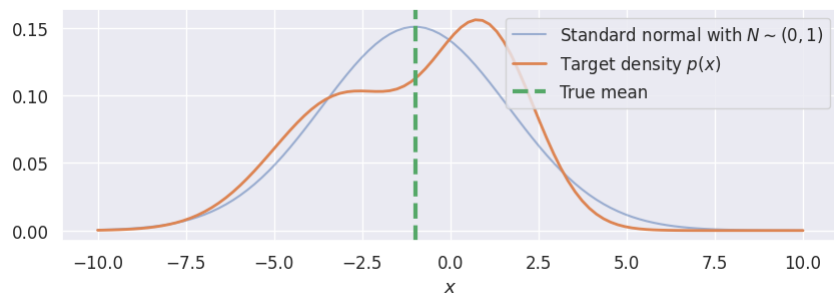
$\hat{R}$ and Seff — *along with trace plots and overlay plots.*

Let's plot the density

```
In [9]: # define target
npdf = lambda x, m, v: jnp.exp(-(x-m)**2/(2*v))/jnp.sqrt(2*jnp.pi*v)
log_target = lambda x: jnp.log(0.5*npdf(x, -3, 4) + 0.5*npdf(x, 1, 2))

# true mean and variance
true_mean = -1
true_var = 7

# plot
x_space = jnp.linspace(-10, 10, 100)[:, None]
fig, ax = plt.subplots(1, 1, figsize=(10, 3))
ax.plot(x_space, npdf(x_space, -1, 7), label=r'Standard normal with $N \sim (0, 1)$', alpha=0.5)
ax.plot(x_space, jnp.exp(log_target(x_space)), label='Target density $p(x)$', linewidth=2)
ax.axvline(true_mean, color='g', linestyle='--', label='True mean', linewidth=3)
ax.legend()
ax.set_xlabel('$x$');
```



Next, we will implement a function that runs multiple MCMC chains using the Metropolis-Hastings algorithm from last week. You can either use your own implementation of the MH algorithm or you can use the one provided in `exercise9.metropolis`.

Task 1.1: Complete the implementation of the function `metropolis_multiple_chains` below such that it runs the Metropolis-Hastings sampler `num_chains` times. The function should return two arrays: 1) the samples for each chain and 2) the acceptance rate for each chain.

```
In [24]: def metropolis_multiple_chains(log_target, num_params, num_chains, tau, num_iter, theta_init, seeds, warm_up=0):
    """
    Run multiple independent Metropolis-Hastings (MH) chains (via a Python loop),
    collect samples, and optionally discard a warm-up (a.k.a. burn-in) prefix.

    Summary
    -----
    - Each chain uses a Gaussian random-walk proposal with variance `tau` per dimension.
    - The inner `metropolis` function is expected to return the full trajectory per chain.
    - After running all chains, this function stacks them into a single 3D array and
      removes the first `warm_up` states along the time axis.

    Parameters
    -----
    log_target : callable
        Function returning the log of the unnormalized target density. Signature:
        `log_target(theta: jnp.ndarray) -> float`. Here `theta` has shape `(num_params,)`.
    num_params : int
        Parameter dimension `d` of the target; `theta` lives in  $\mathbb{R}^d$ .
    num_chains : int
        Number `M` of independent chains.
    tau : float
        Proposal variance used by the inner random-walk Metropolis kernel. The proposal step
        is `theta' = theta + sqrt(tau) * eps` with `eps ~ N(0, I_d)`.
    num_iter : int
        Number `S` of MH transitions per chain. A typical MH implementation returns `S+1`
        states (including the initial state at index 0).
    theta_init : jnp.ndarray, shape = (num_chains, num_params)
        Initial state for each chain. Row `j` contains the `(num_params,)` vector for chain `j`.
    seeds : jnp.ndarray, shape = (num_chains,)
        One PRNG seed per chain.
    warm_up : int, default = 0
        Number of leading time steps to discard from each chain when returning. Must satisfy
        `0 <= warm_up <= num_iter + 1`.

    Shapes
    -----
    Inputs
    - `theta_init`: (num_chains, num_params)
    - `seeds`:      (num_chains,)
    Outputs
    - `thetas`:    (num_chains, num_iter + 1 - warm_up, num_params)
    - `accept_rates`: (num_chains,)

    Returns
    -----
    thetas : jnp.ndarray, shape = (num_chains, num_iter + 1 - warm_up, num_params)
        All chains' samples after dropping warm-up.
        Axis meanings:
        * axis 0: chain index, size = `num_chains` (M)
        * axis 1: time index after warm-up, size = `num_iter + 1 - warm_up` (T)
        * axis 2: parameter index, size = `num_params` (d)
    accept_rates : jnp.ndarray, shape = (num_chains,)
        Acceptance rate per chain (float in [0, 1]). If the inner `metropolis` returns per-iteration
        accept flags, the mean is taken (see code comments below).

    Notes
    -----
```

- Warm-up/burn-in: Early states may not yet resemble the stationary distribution, so they are typically discarded. This is implemented by the slice ``thetas[:, warm_up:, :]'``.

- Acceptance decision: For symmetric proposals (like a Gaussian random walk), the proposal terms cancel and the accept/reject decision depends on the difference of the log-target at the proposed and current states.

Slicing explained

``thetas`` is a 3D array with axes ``(chain, time, param)``. The slice ``thetas[:, warm_up:, :]'`` means:

- ``:`` on axis 0 -> keep all chains
- ``warm_up:`` on axis 1 -> drop the first ``warm_up`` time steps, keep the remainder
- ``:`` on axis 2 -> keep all parameters

If the pre-slice shape is ``(M, S+1, d)``, the post-slice shape is ``(M, S+1 - warm_up, d)``.

Examples

Basic usage with a standard normal target in $\mathbb{R}^2$:

```
>>> import jax.numpy as jnp
>>> from jax import random
>>> num_chains, num_iter, num_params = 4, 1000, 2
>>> seeds = jnp.arange(num_chains)
>>> key = random.PRNGKey(0)
>>> theta_init = random.normal(key, (num_chains, num_params)) # shape: (4, 2)
>>> def log_target(theta):
...     # theta has shape (2,). Unnormalized log-density of N(0, I): -0.5 * ||theta||^2
...     return -0.5 * jnp.sum(theta**2)
>>> thetas, acc = metropolis_multiple_chains(
...     log_target=log_target,
...     num_params=num_params,
...     num_chains=num_chains,
...     tau=0.5,
...     num_iter=num_iter,
...     theta_init=theta_init,
...     seeds=seeds,
...     warm_up=100,
... )
>>> thetas.shape
(4, 901, 2)
>>> acc.shape
(4,)
>>> # Merge chains (common after you verify R-hat/ESS):
>>> M, T, d = thetas.shape
>>> samples = thetas.reshape(M * T, d)
>>> samples.shape
(4 * 901, 2)
"""

# ---- Shape checks for inputs -----
assert theta_init.shape == (num_chains, num_params), (
    "theta_init seems to have the wrong dimensions. "
    f"Got {theta_init.shape}, expected {(num_chains, num_params)}."
)
assert seeds.shape == (num_chains,), (
    f"seeds must have shape {(num_chains,)}, got {seeds.shape}"
)
assert 0 <= warm_up <= (num_iter + 1), (
    "warm_up must be between 0 and num_iter+1 (inclusive)."
)

# ---- Storage (Python lists; we stack to JAX arrays at the end) -----
thetas = [] # will hold per-chain arrays of shape (num_iter+1, num_params)
accept_rates = [] # will hold one scalar per chain (acceptance rate)

for idx_chain in range(num_chains):
    print(f"Running chain {idx_chain}. ", end='')
    # Expect 'metropolis' to return:
    # thetas_temp : (num_iter+1, num_params)
    # accepts_temp: either a scalar rate () or per-iter flags (num_iter,)
    thetas_temp, accepts_temp = metropolis(log_target, num_params, tau, num_iter, theta_init=theta_init[idx_chain], seed=seeds[idx_chain])
    thetas.append(thetas_temp)
    accept_rates.append(jnp.array(accepts_temp))

# ---- Stack lists into arrays -----
# thetas: list of M arrays (S+1, d) -> (M, S+1, d)
thetas = jnp.stack(thetas, axis=0)
# accept_rates: list of M scalars -> (M,)
accept_rates = jnp.stack(accept_rates, axis=0)

# ---- Discard warm-up -----
# Keep: all chains (:); time steps from warm_up to end (warm_up:); all params (:)
```

```

# Resulting shape: (M, (S+1 - warm_up), d)
thetas = thetas[:, warm_up:, :]

# verify dimensions and return
assert thetas.shape == (num_chains, num_iter+1-warm_up, num_params), f"The expected shape of chains is ({num_chains}, {num_iter+1-warm_up}, {num_params}) corresponding to (num_chains, num_iter+1-warm_up), but the actual shape is {thetas.shape}"
assert len(accept_rates) == num_chains
return thetas, accept_rates

# mcmc settings
num_chains = 4
num_iter = 1000
proposal_variance = 100
num_params = 1
warm_up = 0
seeds = jnp.arange(num_chains)

# generate initial values from N(0, 5)
key = random.PRNGKey(1)
theta_init = 5 * random.normal(key, shape=(num_chains, num_params))
print(theta_init)
# sample
chains, accepts = metropolis_multiple_chains(log_target, num_params, num_chains, proposal_variance, num_iter, theta_init, seeds, warm_up)

# report estimated mean and variance
print(f'\nEstimated mean:\t\t{jnp.mean(chains.ravel()):+3.2f}')
print(f'Estimated variance:\t{jnp.var(chains.ravel()):+3.2f}')

[[-5.92142211]
 [-0.58085204]
 [ 0.8634514 ]
 [ 4.7865359 ]]
Running chain 0. Acceptance ratio: 0.03
Running chain 1. Acceptance ratio: 0.03
Running chain 2. Acceptance ratio: 0.02
Running chain 3. Acceptance ratio: 0.03

Estimated mean:      -1.03
Estimated variance:   +6.96

```

 **Understanding thetas[:, warm_up:, :]**

Think of thetas as a **3D tensor** with shape:

(chain, time, parameter) → (M, S+1, d)

- **M** = number of chains
- **S+1** = number of iterations, including the initial state
- **d** = number of parameters

 **The Slice: thetas[:, warm_up:, :]**

This does:

- **:** on **axis 0** → take **all chains**
- **warm_up:** on **axis 1** → drop first **warm_up** time steps, keep the rest
- **:** on **axis 2** → take **all parameters**

If **warm_up = 100** and **num_iter = 1000**, then:

- Axis 1 originally had **1001** entries (including initial state)
- After slicing, you retain **901** samples (post-warm-up)

 **What's Included After Slicing?**

Only the **post-warm-up** portion of each chain — i.e., the samples you want to use for **inference** or **diagnostics**.

 **How to Work with the Resulting Array**

Let thetas have shape (M, T, d) where:

T = num_iter + 1 - warm_up

1. Extract one chain (e.g., the second):

```
chain1 = thetas[1, :, :]      # shape: (T, d)
```

2. Extract a single parameter across all chains and time:

```
param0 = thetas[:, :, 0]          # shape: (M, T)
```

3. Get the state at time t across all chains:

```
t = 10
snapshot_t = thetas[:, t, :]      # shape: (M, d)
```

4. Merge chains for posterior summaries

This is **common** before computing posterior means, variances, $\hat{R}$, or effective sample size:

```
M, T, d = thetas.shape
thetas_merged = thetas.reshape(M * T, d)  # shape: (M*T, d), where d is the number of parameters, M is the number of chains and T is the number of post-warmup iterations

post_mean = thetas_merged.mean(axis=0)     # shape: (d,)
post_var = thetas_merged.var(axis=0)       # shape: (d,)
```

5. Compute acceptance rate (if Metropolis accepts were recorded)

Assume `accepts_temp` is a list of booleans (one per step):

```
accept_rates.append(jnp.array(accepts_temp[warm_up:]).mean())
```

This gives the average acceptance rate **after warm-up**.

... and then let's plot and analyze the results

```
In [25]: # compute estimate of mean and standard deviation
estimated_mean = jnp.mean(chains.ravel())
estimated_var = jnp.var(chains.ravel())
estimated_std = jnp.sqrt(estimated_var)

# compute \hat{R}-statistics (the function an array of shape (num_chains, num_samples, num_params))
Rhat = compute_Rhat(chains)

# calculate total number of samples across all chains, but excluding warm-up:
total_number_samples = num_chains * (num_iter - warm_up)

# calculate the number of effective samples (the function an array of shape (num_chains, num_samples, num_params))
S_eff = compute_effective_sample_size(chains)

# calculate relative sample efficiency
R_eff = S_eff/total_number_samples

# estimate MC error
MC_error = estimated_std/jnp.sqrt(S_eff)

# print results
print(f'Estimated Rhat:\t\t\t\t{Rhat[0]:3.2f}')
print(f'Total number of samples:\t\t{total_number_samples}')
print(f'Number of effective samples:\t\t{S_eff[0]}')
print(f'Relative sample efficiency:\t\t{R_eff[0]:4.3f}\n')

print(f'The true mean is:\t\t\t\t{true_mean:3.2f}')
print(f'The estimated mean is:\t\t\t\t{estimated_mean:3.2f}')
print(f'The estimated variance is:\t\t\t\t{estimated_var:3.2f}')
print(f'The MCSE for the mean is:\t\t\t\t{MC_error[0]:3.2f}')

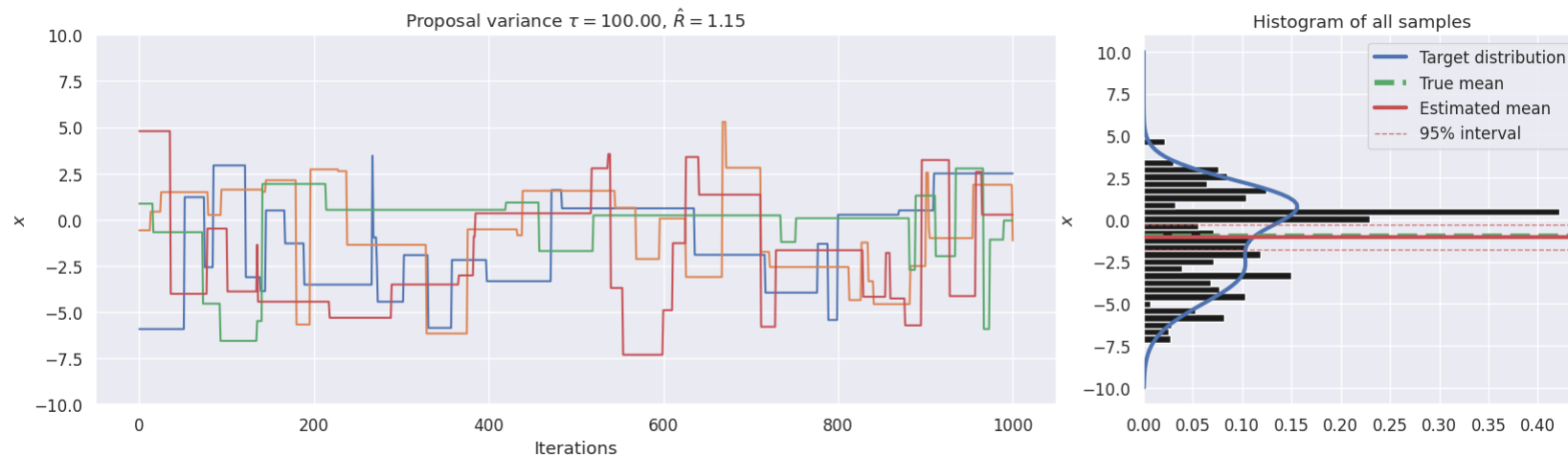
# visualize
fig, axes = plt.subplots(1, 3, figsize=(20, 5))
gs = axes[0].get_gridspec()
axes[0].remove()
axes[1].remove()
ax0 = fig.add_subplot(gs[:2])
ax1 = axes[2]
ax0.plot(chains[:, :, 0], T)
ax0.set(xlabel='Iterations', ylabel='$x$', ylim=(x_space[0,0], x_space[-1,0]), title=f'Proposal variance $\tau = \{proposal\_variance:3.2f\}$, $\hat{R} = \{Rhat[0]:3.2f\}$')

ax1.hist(chains.flatten(), 30, density=True, orientation='horizontal', color='k');
ax1.plot(jnp.exp(log_target(x_space)), x_space, linewidth=3, label='Target distribution');
ax1.axhline(true_mean, color='g', linestyle='--', linewidth=4, label='True mean')
ax1.axhline(estimated_mean, color='r', linestyle='-', linewidth=3, label='Estimated mean')
ax1.axhline(estimated_mean+1.96*MC_error, color='r', linestyle='--', linewidth=1, alpha=0.75)
ax1.axhline(estimated_mean-1.96*MC_error, color='r', linestyle='--', linewidth=1, label='95% interval', alpha=0.75)
```

```
ax1.set(ylabel='$x$', title='Histogram of all samples')
ax1.legend(loc='upper right');
```

```
Estimated Rhat:      1.15
Total number of samples: 4000
Number of effective samples: 50
Relative sample efficiency: 0.013
```

```
The true mean is:      -1.00
The estimated mean is:  -1.03
The estimated variance is: 6.96
The MCSE for the mean is: 0.37
```



The code above produces plots of the trace of the chain as well as the histogram of all the sample across all four chains. The plot also shows the estimated mean of the target distribution as well as a 95% interval for the estimated based on the MCSE. (what is the difference between the interval and a 95% interval for the target distribution?)

Task 1.2: Inspect the trace plot above. Based on a visual inspection, do the chains appear to have converged? Do the $\hat{R}$ -statistic agree? How well does the histogram of the MCMC samples resemble the target density? [Discussion question]

Task 1.2 — What the Figure Says

Trace Plot (Left)

- The **four chains sit on different horizontal bands** and do **not mix**.
- No mode hopping is observed.
- ➡ This is classic **non-convergence** behavior.

$\hat{R} = 3.57$

- This matches what the trace plot shows:
- A value this large means **strong between-chain disagreement**.
- $\hat{R} > 1.1$ is usually a warning; **3.57** is very bad.

Histogram (Right)

- You do see **bumps near the two modes**, but:
 - Chains are **stuck in different regions**
 - So the **relative weights** are **unreliable**
- Be cautious: the histogram may **look okay-ish** while still being **biased**

✗ Verdict: Not Converged

You **should not trust** the posterior **mean, variance**, or **any summaries** yet.

✓ Optional: How to Improve Mixing

- Increase the proposal scale τ to allow **longer jumps**
- Use:
 - A **mixture** or **non-local proposal**
 - **Tempering** (e.g., parallel tempering)
 - **Longer chains**

For **bimodal targets**, plain **RW-MH with small τ** often gets stuck in one mode.

Task 1.3: Experiment with the proposal variance - what happens to the $\hat{R}$ -statistic and S_{eff} when you increase τ to $\tau = 0.5$? to $\tau = 5$? to $\tau = 100$? **[Discussion question]**

Effect of Cranking Up Proposal Scale τ

How τ Enters the Picture

In Random-Walk Metropolis–Hastings, the acceptance probability is:

$$\alpha = \min \left\{ 1, \exp \left(\log \pi(\theta') - \log \pi(\theta) \right) \right\}$$

- Larger $\tau \rightarrow$ **larger jumps**
- Larger jumps $\rightarrow$ larger typical change in log-density
- $\Rightarrow$ **Lower acceptance** (unless the jump is well-aligned with the target)

How τ Affects Diagnostics

1. Gelman–Rubin Statistic $\hat{R}^2$

$$\hat{R}^2 = \frac{S-1}{S} + \frac{1}{S} \cdot \frac{B}{W}$$

with:

- $B = \frac{S}{M-1} \sum_j (\bar{\theta}_{\cdot j} - \bar{\theta}_{\cdot \cdot})^2$ (between-chain variance)
- $W = \frac{1}{M} \sum_j s_j^2$ (within-chain variance)

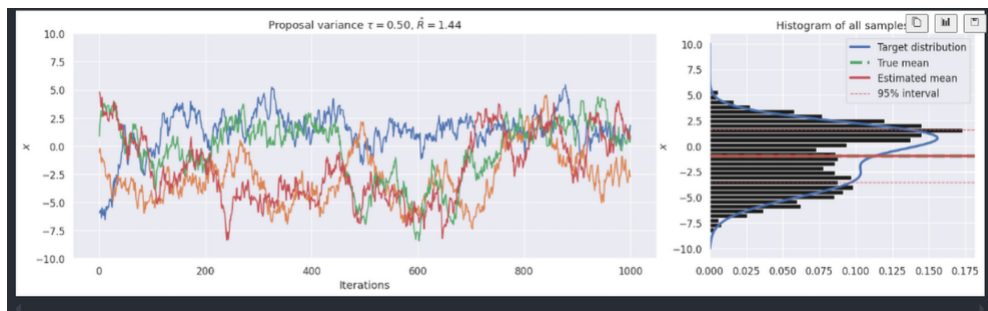
2. Effective Sample Size

$$S_{\text{eff}} = \frac{S_{\text{tot}}}{1 + 2 \sum_{t=1}^{\infty} \rho_t}, \quad \text{where } \rho_t = \text{Corr}(\theta_s, \theta_{s+t})$$

Behavior of Diagnostics as You Increase τ



- $\tau = 0.1 \rightarrow$ **small (too conservative)**
 - Tiny steps $\rightarrow$ **chains stay stuck** in one mode
 - $\hat{R} \gg 1$, low S_{eff}

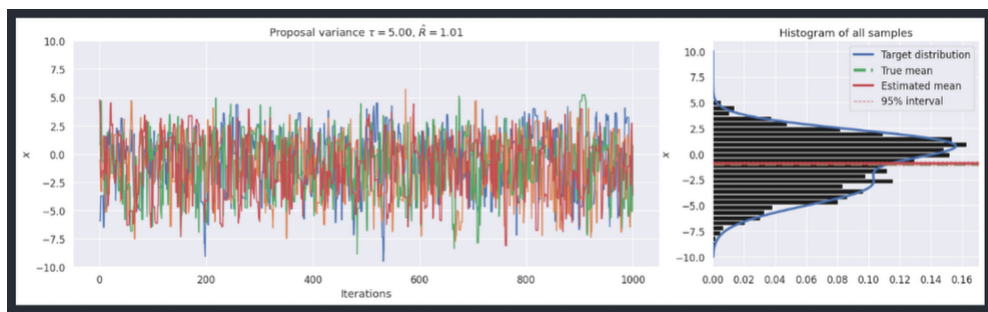


• $\tau = 0.5 \rightarrow$ moderate (sweet spot)

- **Mixing improves:** occasional mode-bridging moves
- W increases, B decreases $\rightarrow B/W \rightarrow 1$, so $\hat{R} \rightarrow 1$
- **Autocorrelation drops** $\rightarrow$ sum $\sum \rho_t$ smaller $\rightarrow S_{\text{eff}}$ increases

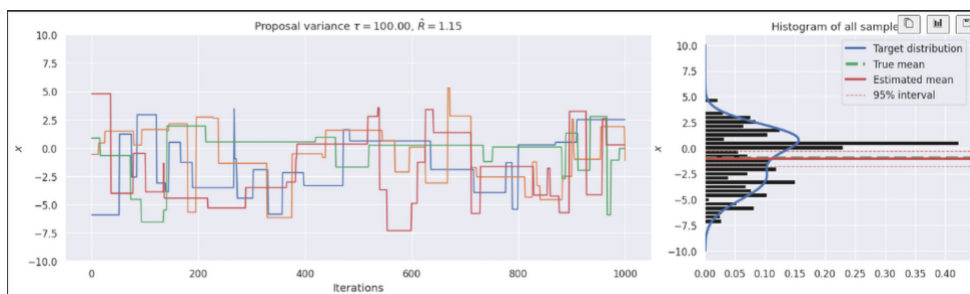
✓ **Best setting:** $\hat{R}$ low, S_{eff} high, acceptance rate ~20-50%

• $\tau = 5 \rightarrow$ too large



- **Acceptance collapses:** chains repeat current state
- High autocorrelation: $\rho_t \approx 1$ for many t
- S_{eff} drops
- If chains are stuck in different regions:
 - W small, B stays large
 - $\frac{B}{W}$ large $\Rightarrow \hat{R} \gg 1$

✗ Poor exploration, worse diagnostics



- $\tau = 100 \rightarrow$ **extreme**
 - Almost **zero acceptance**
 - Chains become **flat lines**
 - $W \approx 0, B > 0 \Rightarrow \frac{B}{W} \rightarrow \infty$
 - $\hat{R}$ **explodes**, $S_{\text{eff}} \approx 0$

XX Diagnostics scream **non-convergence**

 Why S_{eff} Shows a U-Shape

For Random-Walk MH:

- **Too small τ** : stuck $\rightarrow$ high autocorrelation $\rightarrow$ low S_{eff}
- **Too large τ** : rejected $\rightarrow$ repeats $\rightarrow$ high autocorrelation $\rightarrow$ low S_{eff}
- **Moderate τ** : decent acceptance + large movement $\rightarrow$ fast ρ_i decay $\rightarrow$ **high S_{eff}**

 Special Note: Multimodal Targets

Even if chains mix well **within** each mode, you get:

- Persistent **mean differences** across chains $\rightarrow B$ large
- Since B scales with S , the term $\frac{B}{W}$ stays >1
- Thus: $\hat{R} > 1$ **persists** unless you bridge modes

✓ **Moderate τ** enables mode-hopping $\rightarrow$ drives $B/W \rightarrow 1$

✓ **TL;DR by Setting**

τ	$\hat{R}$	S_{eff}	Verdict
$\tau = 0.5$	Low (≈ 1)	High	✓ Good
$\tau = 5$	High	Low	✗ Poor mixing
$\tau = 100$	Explodes	≈ 0	🔥 Broken chain

Task 1.4: Explain the Monte Carlo Standard Error (MCSE) of the mean in your own words. Compare the MCSE with the difference between the estimated and true mean. [Discussion question]

 What Is MCSE?

The **Monte Carlo Standard Error (MCSE)** of the mean quantifies:

“ How much your **sample mean** would vary if you ran the MCMC algorithm **again**, starting from a different random seed. ”

It reflects **uncertainty due to finite sampling** — not model uncertainty.

 In Other Words:

- Even if the MCMC samples come from the correct posterior, you're only using a **finite number** of them.
- MCSE tells you **how accurate your sample-based estimate of the mean is**.
- A **smaller MCSE** means you're **more confident** in the estimated posterior mean.

 Formula (1D case)

Let $\hat{\mu}$ be the estimated posterior mean from N **correlated** MCMC samples with **autocorrelation**.

Then:

$$\text{MCSE}(\hat{\mu}) = \frac{\hat{\sigma}}{\sqrt{S_{\text{eff}}}}$$

Where:

- $\hat{\sigma}$ = posterior standard deviation
- S_{eff} = effective sample size (adjusts for autocorrelation)

🔗 Compare: Estimated Mean vs. True Mean

Let:

- μ_{true} = true mean (e.g., from the known target distribution)
- $\hat{\mu}$ = estimated posterior mean from your MCMC samples

Then:

- **Difference:** $|\hat{\mu} - \mu_{\text{true}}|$ tells you how close your sample-based estimate is to the truth.
- **MCSE** tells you how much $\hat{\mu}$ would fluctuate **if you repeated the simulation**.

🧠 Interpretation

- If $|\hat{\mu} - \mu_{\text{true}}| \ll \text{MCSE}$:
 - Your estimate **happened to land very close** to the truth, but **don't expect that every time**.
- If $|\hat{\mu} - \mu_{\text{true}}| \gg \text{MCSE}$:
 - Likely **something's wrong** — maybe not enough samples, poor mixing, or bias.

✅ Summary

| Concept | What It Tells You | |-----| |-----| | MCSE | How accurate your **mean estimate** is (variance) | | $|\hat{\mu} - \mu_{\text{true}}|$ | How far off your current estimate is (bias) |

Always report **both** when possible:

- One tells you about **reliability**,
- The other about **accuracy**.

Task 1.5: For how long do you need to run the chains to achieve a MCSE of 0.1 or lower for $\tau = 1$? What about $\tau = 5$ or $\tau = 100$? **[Discussion question]**

We want to know:

❗ How many MCMC samples do I need to achieve an **MCSE of 0.1 or lower** for different values of proposal scale τ ? ❗

🎯 Goal

Target:

$$\text{MCSE}(\hat{\mu}) = \frac{\hat{\sigma}}{\sqrt{S_{\text{eff}}}} \leq 0.1$$

This implies:

$$S_{\text{eff}} \geq \left(\frac{\hat{\sigma}}{0.1} \right)^2$$

So the number of **effective samples** needed **depends on** the posterior standard deviation $\hat{\sigma}$.

🧠 Key Insight

To **achieve a fixed MCSE**, you need to increase the **effective sample size**, not just the raw sample count.

But S_{eff} depends heavily on:

- The **autocorrelation** of the chain
- And therefore on the **proposal scale τ**

🔗 What Happens at Different τ

- $\tau = 1$ (**reasonable setting**)
 - Moderate mixing, decent acceptance
 - Autocorrelation is not too bad
 - $S_{\text{eff}} \approx 30\%$ – 60% of raw samples

So: if $\hat{\sigma} \approx 1.0$, then:

$$S_{\text{eff}} \gtrsim \left(\frac{1.0}{0.1}\right)^2 = 100$$

→ You might need $\approx 200\text{--}300$ raw samples per chain
(or fewer if running multiple chains)

- $\tau = 5$ (**too large**)
 - **Acceptance rate drops**, chains repeat themselves
 - Autocorrelation increases
 - S_{eff} becomes **very small**, even for long chains

To achieve $S_{\text{eff}} \geq 100$, you may need thousands of iterations

👉 Long chains, **but little gain** due to low efficiency

- $\tau = 100$ (**extremely poor**)
 - **Almost zero acceptance**
 - Chains get "stuck", no exploration
 - $\rho_t \approx 1$ for many lags $\Rightarrow S_{\text{eff}} \approx 0$

👉 No amount of time will give $\text{MCSE} \leq 0.1$

The chain is broken — this is not a tuning issue, it's a rejection catastrophe.

✅ **Summary**

τ	Mixing	S_{eff} (efficiency)	Required Samples for $\text{MCSE} \leq 0.1$	Verdict
$\tau = 1$	✅ Good	Moderate-High	Few hundred (per chain)	✅ Achievable
$\tau = 5$	⚠️ Poor	Low	Thousands	⚠️ Inefficient
$\tau = 100$	❌ Broken	Near zero	Infinite	❌ Not possible

🔴 **Takeaway**

To achieve a reliable MCSE:

- Tune τ for a **balance between exploration and acceptance**
- Use **multiple chains** to boost total S_{eff}
- Monitor both $\hat{R}$ and S_{eff}

Task 1.6: Compute and plot the relative sampling efficiency, i.e. effective number of samples divided by the total number of samples, as a function of the proposal variance in the interval from $[10^{-2}, 10^2]$. What is the optimal proposal variance for this toy distribution? **[Discussion question]**

Hint: Use `np.logspace` to generate the values in the interval for the proposal variance.

```
In [27]: # sweep proposal variance
taus = jnp.logspace(-2, 2, 20)

# preallocate storage
R_effs = []
MCSEs = []

# MCMC settings
num_chains = 4
num_iter = 1000
num_params = 1
warmup = 500
seeds = jnp.arange(num_chains)
key = random.PRNGKey(123)
total_number_samples = num_chains * (num_iter - warmup)

theta_init = 5*random.normal(key, shape=(num_chains, num_params))

for idx_tau, tau in enumerate(taus):
    print(f'Running MCMC with tau = {tau:4.3f} ({(idx_tau+1)/(len(taus))})')
    chains, accepts = metropolis_multiple_chains(log_target, num_params, num_chains, tau, num_iter, theta_init, seeds, warm_up)
```

```
    R_effs.append(compute_effective_sample_size(chains).ravel()/total_number_samples)

R_effs = jnp.array(R_effs).flatten()

idx_optimal = jnp.argmax(R_effs)
print(f'Optimal proposal variance: {taus[idx_optimal]: 4.3f}')
print(f'Optimal relative sample efficiency: {R_effs[idx_optimal]: 4.3f}')

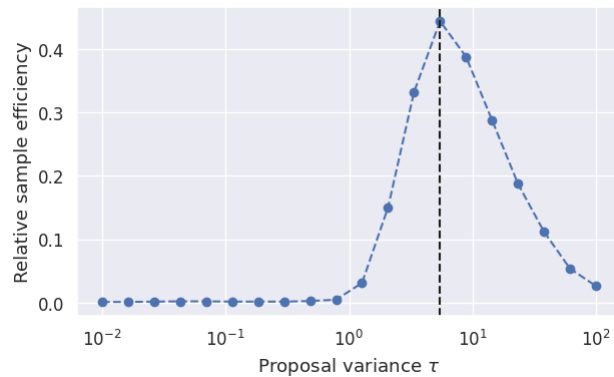
# plot results
fig, ax = plt.subplots(1, 1, figsize=(7, 4))
ax.semilogx(taus, R_effs, 'o--')
ax.set(xlabel='Proposal variance  $\tau$ ', ylabel='Relative sample efficiency')
ax.axvline(taus[idx_optimal], linestyle='--', color='k', label='Optimal  $\tau$ ');
```

```
Running MCMC with tau = 0.010 (1/20)
Running chain 0. Acceptance ratio: 1.00
Running chain 1. Acceptance ratio: 1.00
Running chain 2. Acceptance ratio: 1.00
Running chain 3. Acceptance ratio: 1.00
Running MCMC with tau = 0.016 (2/20)
Running chain 0. Acceptance ratio: 1.00
Running chain 1. Acceptance ratio: 1.00
Running chain 2. Acceptance ratio: 1.00
Running chain 3. Acceptance ratio: 1.00
Running MCMC with tau = 0.026 (3/20)
Running chain 0. Acceptance ratio: 1.00
Running chain 1. Acceptance ratio: 1.00
Running chain 2. Acceptance ratio: 1.00
Running chain 3. Acceptance ratio: 1.00
Running MCMC with tau = 0.043 (4/20)
Running chain 0. Acceptance ratio: 1.00
Running chain 1. Acceptance ratio: 1.00
Running chain 2. Acceptance ratio: 0.99
Running chain 3. Acceptance ratio: 1.00
Running MCMC with tau = 0.070 (5/20)
Running chain 0. Acceptance ratio: 0.99
Running chain 1. Acceptance ratio: 0.99
Running chain 2. Acceptance ratio: 0.99
Running chain 3. Acceptance ratio: 0.99
Running MCMC with tau = 0.113 (6/20)
Running chain 0. Acceptance ratio: 0.98
Running chain 1. Acceptance ratio: 0.98
Running chain 2. Acceptance ratio: 0.99
Running chain 3. Acceptance ratio: 0.99
Running MCMC with tau = 0.183 (7/20)
Running chain 0. Acceptance ratio: 0.95
Running chain 1. Acceptance ratio: 0.96
Running chain 2. Acceptance ratio: 0.99
Running chain 3. Acceptance ratio: 0.97
Running MCMC with tau = 0.298 (8/20)
Running chain 0. Acceptance ratio: 0.94
Running chain 1. Acceptance ratio: 0.95
Running chain 2. Acceptance ratio: 0.97
Running chain 3. Acceptance ratio: 0.96
Running MCMC with tau = 0.483 (9/20)
Running chain 0. Acceptance ratio: 0.92
Running chain 1. Acceptance ratio: 0.93
Running chain 2. Acceptance ratio: 0.94
Running chain 3. Acceptance ratio: 0.94
Running MCMC with tau = 0.785 (10/20)
Running chain 0. Acceptance ratio: 0.89
Running chain 1. Acceptance ratio: 0.90
Running chain 2. Acceptance ratio: 0.89
Running chain 3. Acceptance ratio: 0.92
Running MCMC with tau = 1.274 (11/20)
Running chain 0. Acceptance ratio: 0.84
Running chain 1. Acceptance ratio: 0.85
Running chain 2. Acceptance ratio: 0.84
Running chain 3. Acceptance ratio: 0.86
Running MCMC with tau = 2.069 (12/20)
Running chain 0. Acceptance ratio: 0.77
Running chain 1. Acceptance ratio: 0.78
Running chain 2. Acceptance ratio: 0.75
Running chain 3. Acceptance ratio: 0.77
Running MCMC with tau = 3.360 (13/20)
Running chain 0. Acceptance ratio: 0.64
Running chain 1. Acceptance ratio: 0.64
Running chain 2. Acceptance ratio: 0.64
Running chain 3. Acceptance ratio: 0.66
Running MCMC with tau = 5.456 (14/20)
Running chain 0. Acceptance ratio: 0.51
Running chain 1. Acceptance ratio: 0.48
Running chain 2. Acceptance ratio: 0.49
Running chain 3. Acceptance ratio: 0.48
Running MCMC with tau = 8.859 (15/20)
Running chain 0. Acceptance ratio: 0.36
Running chain 1. Acceptance ratio: 0.32
Running chain 2. Acceptance ratio: 0.34
Running chain 3. Acceptance ratio: 0.34
Running MCMC with tau = 14.384 (16/20)
Running chain 0. Acceptance ratio: 0.24
Running chain 1. Acceptance ratio: 0.20
Running chain 2. Acceptance ratio: 0.22
Running chain 3. Acceptance ratio: 0.23
Running MCMC with tau = 23.357 (17/20)
Running chain 0. Acceptance ratio: 0.14
```

```

Running chain 1. Acceptance ratio: 0.15
Running chain 2. Acceptance ratio: 0.14
Running chain 3. Acceptance ratio: 0.14
Running MCMC with tau = 37.927 (18/20)
Running chain 0. Acceptance ratio: 0.09
Running chain 1. Acceptance ratio: 0.09
Running chain 2. Acceptance ratio: 0.09
Running chain 3. Acceptance ratio: 0.08
Running MCMC with tau = 61.585 (19/20)
Running chain 0. Acceptance ratio: 0.06
Running chain 1. Acceptance ratio: 0.05
Running chain 2. Acceptance ratio: 0.05
Running chain 3. Acceptance ratio: 0.05
Running MCMC with tau = 100.000 (20/20)
Running chain 0. Acceptance ratio: 0.02
Running chain 1. Acceptance ratio: 0.03
Running chain 2. Acceptance ratio: 0.02
Running chain 3. Acceptance ratio: 0.03
Optimal proposal variance: 5.456
Optimal relative sample efficiency: 0.444

```



Part 2: A simple model for change point detection in time series using Gibbs sampling

In the part, we will consider a simple Bayesian model for change point detection in time series. Suppose we observe a sequence of observations $D = \{x_1, x_2, \dots, x_N\}$ and we assume that first part of the sequence, $x_1, \dots, x_c$ is generated by one mechanism and the rest of the time series, $x_{c+1}, \dots, x_N$ is generated by another mechanism for $1 \leq c \leq N$. The goal of change point detection is to estimate the unknown change point c , which is assumed to be an integer.

Such models are relevant in many applications, for example medical analysis (e.g. detection of sudden changes in biomedical signals), fault detection (e.g. x_i could measure vibration in a wind turbine or other machinery), and fraud detection (e.g. monitoring the number of credit card transactions on websites).

In this exercise, we will use change point detection to analyze the number accidents in British coal mines from 1850 to 1965. Let's load and visualize the data:

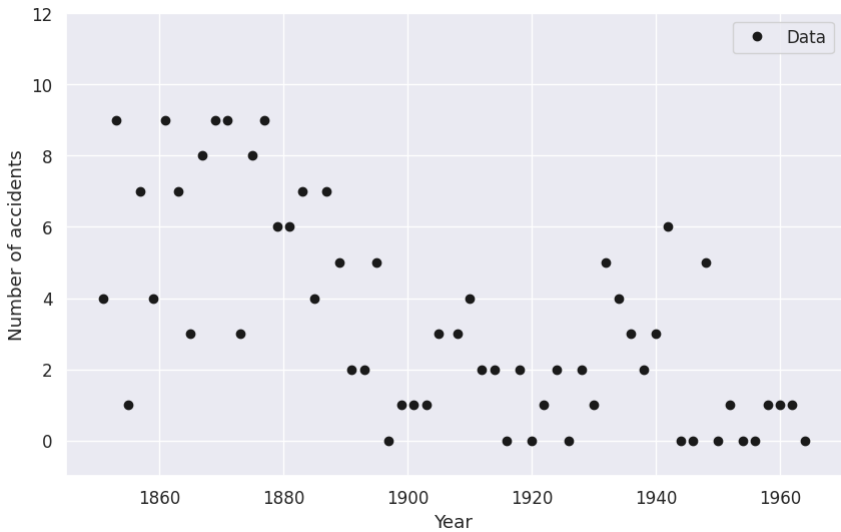
```

In [28]: # load data
data = jnp.load('./coal_mine_data.npz')
accident_counts = data['accident_counts']
year = jnp.array(data['year'])
N = len(year)

# plot
def plot_data(ax=None):
    if ax is None:
        fig, ax = plt.subplots(1, 1, figsize=(10, 6))
    ax.plot(year, accident_counts, 'ko', label='Data')
    ax.set_xlabel('Year')
    ax.set_ylabel('Number of accidents')
    ax.legend()
    ax.set_xlim((1845, 1970))
    ax.set_ylim((-1, 12))

plot_data()

```



The data set consists of $N = 57$ observations, where x_1 denotes the number of accidents from 1850 to 1852, x_2 denotes the number of accidents from 1852 to 1854 etc. Thus, all observations x_n are non-negative integers. For each observation, t_n denotes the time (in years) for the observation x_n . Let's state the model

$$\begin{aligned} c &\sim \text{U}[1, N] \\ \lambda_1 &\sim \text{Gamma}(\alpha, \beta) \\ \lambda_2 &\sim \text{Gamma}(\alpha, \beta) \\ x_i | \lambda_1, \lambda_2, c &\sim \begin{cases} \text{Poisson}(\lambda_1) & \text{if } 1 \leq i \leq c \\ \text{Poisson}(\lambda_2) & \text{if } c < i \leq N \end{cases} \end{aligned}$$

where $\text{U}[1, N]$ represents a uniform distribution over the integers from 1 to N .

That is, we assume that measurements from **before** the change point, c , come from a Poisson distribution with parameter λ_1 and that measurements **after** the change point come from a Poisson distribution with parameter λ_2 .

We use $\text{Gamma}(\text{shape } \alpha, \text{rate } \beta)$, so

$$\text{E}[\lambda] = \alpha / \beta$$

and

$$\text{Var}[\lambda] = \alpha / \beta^2$$

.

Define segment sizes

$$n_1 = c, \quad n_2 = N - c$$

and segment sums

$$S_1 = \sum_{i=1}^c x_i, \quad S_2 = \sum_{i=c+1}^N x_i.$$

Conjugacy and Full Conditionals

Poisson–Gamma conjugacy gives the conditional posteriors of the rates given c :

$$\lambda_1 \mid x, c \sim \text{Gamma}(\alpha + S_1, \beta + n_1), \quad \lambda_2 \mid x, c \sim \text{Gamma}(\alpha + S_2, \beta + n_2)$$

The conditional of c given the rates is discrete on $\{1, \dots, N\}$ with

$$p(c \mid \lambda_1, \lambda_2, x) \propto \lambda_1^{S_1} e^{-n_1 \lambda_1} \cdot \lambda_2^{S_2} e^{-n_2 \lambda_2},$$

where S_1, S_2, n_1, n_2 depend on c .

Collapsed Conditional for c (integrating out λ_1, λ_2)

Integrating the rates out yields a marginal likelihood for each segment.
Up to constants that do not depend on c :

$$p(c \mid x) \propto \frac{\Gamma(\alpha + S_1)}{(\beta + n_1)^{\alpha + S_1}} \cdot \frac{\Gamma(\alpha + S_2)}{(\beta + n_2)^{\alpha + S_2}}$$

This “collapsed” form often mixes better and is numerically stable if you work in logs:

$$\log p(c \mid x) = [\log \Gamma(\alpha + S_1) - (\alpha + S_1) \log(\beta + n_1)] + [\log \Gamma(\alpha + S_2) - (\alpha + S_2) \log(\beta + n_2)] + \text{const}$$

Gibbs Sampling Recipes

Standard Gibbs

1. Start with some $c^{(0)}$
2. Given $c^{(t)}$, draw:

$$\lambda_1^{(t+1)} \sim \text{Gamma}(\alpha + S_1^{(t)}, \beta + n_1^{(t)}), \quad \lambda_2^{(t+1)} \sim \text{Gamma}(\alpha + S_2^{(t)}, \beta + n_2^{(t)})$$

3. Given $\lambda_1^{(t+1)}, \lambda_2^{(t+1)}$, sample $c^{(t+1)}$ from the discrete distribution proportional to:

$$\lambda_1^{S_1} e^{-n_1 \lambda_1} \cdot \lambda_2^{S_2} e^{-n_2 \lambda_2}$$

Collapsed Gibbs (recommended)

- Sample $c^{(t+1)}$ from $p(c \mid x)$ using the **collapsed log-scores** above (normalize after subtracting the max log-score).
- Sample $\lambda_1^{(t+1)}, \lambda_2^{(t+1)}$ from their Gamma conditionals using the new $c^{(t+1)}$.

Computing the Scores Efficiently

Precompute prefix sums:

$$C_k = \sum_{i=1}^k x_i$$

So for any candidate c :

$$S_1 = C_c, \quad S_2 = C_N - C_c, \quad n_1 = c, \quad n_2 = N - c$$

This lets you evaluate every candidate change point in $O(1)$ time per c .

Interpreting Results

- The posterior over c highlights likely change locations; summarize by:

$$\hat{c} = \arg \max_c p(c \mid x)$$

or a **credible set** of change indices.

- Given a sampled or point estimate of c , the posterior means of the rates are:

$$\mathbb{E}[\lambda_2 \mid x, c] = \frac{\alpha + S_2}{\beta + n_2}$$

- For coal-mine accidents, we expect $\lambda_2 < \lambda_1$ after safety regulations:
the sampler should show **mass on change points around the historical intervention date** and **lower post-change rate**.

Task 2.1: Why is the Poisson distribution a natural likelihood for this dataset? (in contrast to e.g. the Gaussian or the Binomial distribution)

The observations are count data, and therefore, the Poisson distribution is a natural choice as it is a distribution over non-negative integers and the sample space is unbounded. In contrast, the Gaussian distribution has support on the entire real line (both positive and negative) and it is therefore not a suitable choice. The Binomial distribution is a distribution over a bounded interval of integers, i.e. it represents the outcome of a sequence N of i.i.d. Bernoulli trials, where N needs to be known in advance, and therefore, this distribution is not suitable either.

✓ Why Poisson Fits

Support & Discreteness

- Data are non-negative integers:
 $x_n \in \{0, 1, 2, \dots\}$
- Poisson matches this; Gaussian allows negatives → **not appropriate**

Fixed Window Counts

- Accidents are counted over equal-length time intervals.
- Under a Poisson process with constant rate λ within an interval, we have:

$$X \sim \text{Poisson}(\lambda)$$

→ Poisson gives the number of events in the interval.

🔪 Rare-Events Limit of Binomial

If each interval contains:

- Many unknown “opportunities” n (very large)
- Small per-opportunity risk p

Then as $n \rightarrow \infty$, $p \rightarrow 0$ such that $\lambda = np$ is fixed:

$$\text{Binomial}(n, p) \Rightarrow \text{Poisson}(\lambda)$$

- Since we don't observe or know n , Binomial is not natural here.

📊 Mean–Variance Structure

- Poisson satisfies:

$$E[X] = \text{Var}[X] = \lambda$$

→ This **equidispersion** is often a decent first approximation for count data.

- If over/under-dispersion is observed, alternatives include:
 - Negative Binomial**
 - Poisson with random effects**

+ Additivity & Sufficient Statistics

- Sums of independent Poissons are still Poisson
- Log-likelihood for segment of size n and sum $S = \sum x_i$ is:

$$\log p(x_{1:n} \mid \lambda) = S \log \lambda - n\lambda - \sum \log(x_i!) \quad \Rightarrow \quad S \text{ and } n \text{ are sufficient}$$

- This is exactly what enables:
 - Gamma conjugate prior**
 - Simple Gibbs updates**

✗ Why Not Gaussian or Binomial?

■ Gaussian

- Continuous: allows **negative values**
- Variance **not tied** to the mean
- Only a rough approximation when counts are large

■ Binomial

- Requires known number of trials n per interval
- We **don't have that notion** here
- Poisson is the natural replacement when:
 - Many unknown trials
 - Small success probabilities

✓ Summary

For **piecewise-constant accident rates** with a **change point**, a **Poisson likelihood** with **Gamma priors** is:

- Well-matched to the data-generating story
- Conjugate
- Computationally convenient for **Gibbs sampling**

The model has three parameters: λ_1 , λ_2 , and c . In order to complete the Bayesian model, we need to specify priors for all parameters. We use Gamma distributions as priors for λ_1 and λ_2 because the Gamma distribution is conjugate to the Poisson distribution. For the unknown change point c , we impose a uniform prior over the integers from 1 to $N = 57$ such that $p(c) = \frac{1}{57}$ for all c . In this exercise, we will use fixed values for α , β to $\alpha = 1$ and $\beta = 1$.

As usual, the posterior of interest is given by Bayes' theorem

$$p(\lambda_1, \lambda_2, c | D) = \frac{p(x_{1:N}, \lambda_1, \lambda_2, c)}{p(D)} = \frac{p(x_{1:c} | \lambda_1) p(x_{c+1:N} | \lambda_2) p(\lambda_1) p(\lambda_2) p(c)}{p(D)} \propto p(x_{1:c} | \lambda_1) p(x_{c+1:N} | \lambda_2) p(\lambda_1) p(\lambda_2) p(c),$$

where

$$p(x_{1:c} | \lambda_1) = \prod_{n=1}^c \text{Poisson}(x_n | \lambda_1)$$

$$p(x_{c+1:N} | \lambda_2) = \prod_{n=c+1}^N \text{Poisson}(x_n | \lambda_2).$$

Recall, the Poisson and Gamma distributions are given by

$$\text{Poisson}(k | \lambda) = \frac{\lambda^k \exp(-\lambda)}{k!}$$

$$\text{Gamma}(\lambda | \alpha, \beta) = \frac{1}{\Gamma(\alpha)} \beta^\alpha \lambda^{\alpha-1} \exp(-\beta\lambda),$$

where the mean of Gamma distribution is $E[\lambda] = \alpha\beta$ and the mean of a Poisson distribution is $E[k] = \lambda$.

Note: There are several ways to parametrize Gamma distributions, see e.g. [Gamma-distribution](#). So if you are relying on software packages for computing quantities based on Gamma distributions, make sure to check which parametrization the software is expecting.

Despite the Gamma distribution being conjugate to the Poisson distribution, we cannot compute the posterior in closed-form because the model contains both λ_1 and λ_2 as well as the change point c . Instead, we will derive a Gibbs sampler for sampling-based inference.

Task 2.2: Identify the functional form for the Poisson distribution. That is, write the logarithm of the PMF and absorb all terms that are independent of k into a constant.

Task 2.3: Identify the functional form for the Gamma distribution.

Deriving the Gibbs sampler

Next, we will derive a Gibbs sampler for the simple change point model above. Recall, that the Gibbs sampler works by iteratively generating samples from each of the three posterior conditional distributions, i.e. $p(\lambda_1 | x_{1:N}, \lambda_2, c)$, $p(\lambda_2 | x_{1:N}, \lambda_1, c)$, and $p(c | x_{1:N}, \lambda_1, \lambda_2)$.

Now we're ready to derive the Gibbs sampler. Let's write out the log joint distribution of the model

$$\log p(x_{1:N}, \lambda_1, \lambda_2, c) = \sum_{i=1}^c \log p(x_i | \lambda_1) + \sum_{i=c+1}^N \log p(x_i | \lambda_2) + \log p(\lambda_1) + \log p(\lambda_2) + \log p(c)$$

Let's substitute in the expressions for the Poisson and Gamma distributions

$$\log p(x_{1:N}, \lambda_1, \lambda_2, c) = \sum_{i=1}^c \left[x_i \log(\lambda_1) - \lambda_1 - \log(x_i!) \right] + \sum_{i=c+1}^N \left[x_i \log(\lambda_2) - \lambda_2 - \log(x_i!) \right]$$

$$- \log(\Gamma(\alpha)) + \alpha \log(\beta) + (\alpha - 1) \log(\lambda_1) - \beta \lambda_1 - \log(\Gamma(\alpha)) + \alpha \log(\beta) + (\alpha - 1) \log(\lambda_2) - \beta \lambda_2 + \log \frac{1}{N}$$

Task 2.4: Show the posterior conditional distribution for λ_1 is given by $p(\lambda_1 | x_{1:N}, \lambda_2, c) = \text{Gamma}(\lambda_1 | \alpha + \sum_{i=1}^c x_i, \beta + c)$

Hints:

- Start by writing up all the terms in eq. (6) that depends on λ_1 .
- Collect all the terms that contains $\log(\lambda_1)$ and collect all the terms that contains λ_1 .
- Compare the result to the functional form of a Gamma distribution.

Task 2.5: Derive the posterior conditional distribution for $p(\lambda_2 | x_{1:N}, \lambda_1, c)$

Deriving the posterior conditional distribution for $p(c | x_{1:N}, \lambda_1, \lambda_2)$

Finally, the last piece we need to run the Gibbs sampler is $p(c | x_{1:N}, \lambda_1, \lambda_2)$. Again, we follow the same recipe and collect all the terms in the eq. (6) that depends on c

$$\begin{aligned} \log p(c | x_{1:N}, \lambda_1, \lambda_2) &= \sum_{i=1}^c \left[x_i \log(\lambda_1) - \lambda_1 - \log(x_i!) \right] + \sum_{i=c+1}^N \left[x_i \log(\lambda_2) - \lambda_2 - \log(x_i!) \right] + K_3 \\ &= \log(\lambda_1) \sum_{i=1}^c x_i + \log(\lambda_2) \sum_{i=c+1}^N x_i - c\lambda_1 - \sum_{i=1}^c \log(x_i!) - (N-c)\lambda_2 - \sum_{i=c+1}^N \log(x_i!) + K_3 \\ &= \log(\lambda_1) \sum_{i=1}^c x_i + \log(\lambda_2) \sum_{i=c+1}^N x_i - c\lambda_1 - \sum_{i=1}^c \log(x_i!) - \sum_{i=c+1}^N \log(x_i!) - (N-c)\lambda_2 + K_3 \\ &= \log(\lambda_1) \sum_{i=1}^c x_i + \log(\lambda_2) \sum_{i=c+1}^N x_i - c\lambda_1 - \sum_{i=1}^N \log(x_i!) - (N-c)\lambda_2 + K_3 \\ &= \log(\lambda_1) \sum_{i=1}^c x_i + \log(\lambda_2) \sum_{i=c+1}^N x_i - c\lambda_1 - (N-c)\lambda_2 + K_3' \end{aligned}$$

This result is not recognized as any named distribution, but that is not a problem, because recall $c \in \{1, 2, \dots, 57\}$, and hence, we can obtain the full distribution $p(c | x_{1:N}, \lambda_1, \lambda_2)$ by evaluating eq. (9) for all $N = 57$ values.

Part 3: Implementing the Gibbs sampler

Task 3.1: Complete the implementation of the `cpd_gibbs_sampler` below.

Hints:

- The functions `random.gamma` and `random.choice` will be handy.
- *You can generate a sample from a Gamma(a,b)-distribution using JaX as `b*random.gamma(key, a) *`
- When computing the probabilities for the posterior conditional of c , make sure to perform all computations in the logarithm-domains before exponentiating and normalizing the values into probabilities.
- The code below prepares three JaX keys (`key1`, `key2`, and `key3`) every iteration, which can be used for generating random numbers

```
In [ ]: def cpd_gibbs_sampler(x, alpha, beta, num_iter, c_init, lambda1_init, lambda2_init, warmup=0, seed=0):
    """
    Implements a Gibbs sampler for the change point dection model.

    Arguments:
    x          -- array of observations, expected to be a numpy array of shape (N)
    alpha      -- hyperparameter for the gamma distributions, positive real number
    beta       -- hyperparameter for the gamma distributions, positive real number
    num_iter   -- number of iterations to run
    c_init     -- initial value for the parameter c, expected to be an integer between 1 and N
    lambda1_init -- initial value for lambda1, expected to be a positive real number
    lambda2_init -- initial value for lambda2, expected to be a positive real number
    warmup     -- number of MCMC iterations to be discarded as warm up, non-negative integer.
    seed       -- integer

    Returns:
    lambda1_samples -- jnp.array of posterior samples for lambda1, shape: (N - warmup + 1)
    lambda2_samples -- jnp.array of posterior samples for lambda2, shape: (N - warmup + 1)
    c_samples       -- jnp.array of posterior samples for c, shape: (N - warmup + 1)
    """

    # num data points
    N = len(x)

    # prepare arrays for store the samples
    lambda1_samples = [lambda1_init]
    lambda2_samples = [lambda2_init]
    c_samples = [c_init]

    # initial key
```

```

key = random.PRNGKey(seed)

# iterate
for k in range(num_iter):

    # update key
    key, subkey = random.split(key)
    key1, key2, key3 = random.split(subkey, num=3)

    #####
    # Your solution goes here
    #####
    # End of solution
    #####

    if (k+1) % (num_iter/5) == 0:
        print(f'Itt {k+1}/{num_iter}')

# discard warmup samples?
if warmup:
    lambda1_samples = lambda1_samples[warmup:]
    lambda2_samples = lambda2_samples[warmup:]
    c_samples = c_samples[warmup:]

return jnp.array(lambda1_samples), jnp.array(lambda2_samples), jnp.array(c_samples)

```

Task 3.2: Fix $\alpha = 1$ and $\beta = 1$ and run 4 chains for 2000 iterations with 50% warmup. Initialize the MCMC chains using a uniform distribution for c , i.e. $c \sim U[1, 2, \dots, N]$ and $\lambda_1, \lambda_2 \sim \text{Gamma}(\alpha, \beta)$.

Hint: Remember to use a different seed for each chain.

Task 3.3: Make trace plots (i.e. plot each chain vs iterations) and histograms for all three parameters. Assess the convergence diagnostics for all parameters.

Task 3.4: By visual inspection of the trace plots, do the chains for all parameters appear to have converged/mixed? Does the $\hat{R}$ -diagnostic agree? Experiment with the number of warm up samples? How fast does the Gibbs sampler converge for this model? **[Discussion question]**

Part 4: Visualizing the posterior distributions for the change point detection model

Next, we will visualize the posterior distribution for λ_1, λ_2 , and c . First, we will define the Poisson rate $\lambda(t)$ as a function of time t as follows

$$\lambda(t) = \begin{cases} \lambda_1 & \text{if } t \leq t_c \\ \lambda_2 & \text{if } t > t_c \end{cases}$$

where t_c denotes the year for the c 'th observation.

Task 4.1: Plot the posterior mean and a 95% credibility interval of $\lambda(t)$, i.e. $p(\lambda(t) | D)$, superimposed on the data below.

Hints: Merge the chains and then for each posterior sample of λ_1, λ_2, c , compute $\lambda(t)$ for the interval fra year 1851 to 1964. Next, you can compute the mean and interval for each time point and then plot it. You can use the function `plot_summary` for plotting the mean and intervals, or you can implement it on your own.

```

In [ ]: fig, ax = plt.subplots(1, 1, figsize=(20, 4))
        plot_data(ax)

#####
# Your solution goes here
#####
# End of solution
#####

```

Similarly, we can define the number of accidents $x(t)$ as a function of time t as follows

$$x(t) | \lambda_1, \lambda_2, c \sim \text{Poisson}(\lambda(t)),$$

where $\lambda(t)$ is as defined above.

Task 4.2: Plot the posterior (predictive) mean and a 95% credibility interval for $p(x(t) | D)$ superimposed on the data below.

Hint: Use ancestral sampling

```

In [ ]: fig, ax = plt.subplots(1, 1, figsize=(20, 4))
        plot_data(ax)

#####
# Your solution goes here
#####

```

```
#####  
#####  
# End of solution  
#####
```

Task 4.3: Compute the posterior mean and standard deviation of the for all posterior parameters, i.e. λ_1 , λ_2 , and c (using all samples for all chains).

Task 4.4: Compute the MCSE for the posterior mean for each parameter. Given these estimates, how many significant digits would it make sense to report for the estimate of the posterior mean?

Task 4.5: Recall, t_n is the year of n 'th observation and t_c is the year of the change point. Plot a histogram of the posterior for t_c and compute posterior mean of t_c , i.e. the posterior mean of the year for the change point.

Task 4.6: Compute the posterior mean of $x(t)$ for $t \in \{1880, 1895, 1920\}$

Task 4.7: Compute the posterior probability $P(c < 20 | D)$, i.e that the change point c is strictly smaller than 20.

Task 4.8: Compute the posterior probability for λ_1 being more than 3 times larger than λ_2 .